```
# Pin trl FIRST, then install unsloth — this avoids the version conflict
!pip install -q "trl>=0.18.2,<=0.24.0"
!pip install -q "unsloth[colab-new] @ git+https://github.com/unslothai/unsloth.git"
!pip install -q --upgrade transformers accelerate peft
!pip install -q pymupdf python-docx jiwer rapidfuzz nltk datasets
!apt-get install -qq poppler-utils

import nltk
nltk.download("punkt",     quiet=True)
nltk.download("punkt_tab", quiet=True)
nltk.download("stopwords", quiet=True)
print("✅ All dependencies installed")
```

```
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
✅ All dependencies installed
```

```
import os, gc, json, random, re, difflib, shutil, time
from pathlib import Path
import cv2, numpy as np, torch
from PIL import Image, ImageEnhance, ImageFilter
from tqdm.auto import tqdm

from google.colab import drive
drive.mount("/content/drive", force_remount=True)

# ── YOUR ACTUAL DRIVE PATHS ──────────────────────────────
PDF_DIR  = Path("/content/drive/MyDrive/print_pdf")   # x-train PDFs
DOCX_DIR = Path("/content/drive/MyDrive/Print")       # y-train DOCXs

# ── Local workspace ──────────────────────────────────────
BASE        = Path("/content/RenAIssance")
IMAGES_DIR  = BASE / "images"
DATASET_DIR = BASE / "dataset"
CHECKPOINTS = BASE / "checkpoints"
OUTPUT_DIR  = BASE / "outputs"

for d in [IMAGES_DIR, DATASET_DIR, CHECKPOINTS, OUTPUT_DIR,
          DATASET_DIR / "x_train_images",
          DATASET_DIR / "x_train_images_aug"]:
    d.mkdir(parents=True, exist_ok=True)

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

IS_BF16 = torch.cuda.is_bf16_supported()

# Verify source folders
for label, folder in [("PDFs  (x-train)", PDF_DIR),
                      ("DOCXs (y-train)", DOCX_DIR)]:
    files = list(folder.iterdir()) if folder.exists() else []
    print(f"  {label}: {len(files)} files in {folder}")
    for f in files:
        print(f"    {f.name}")
print("\n✅ Config ready")
```

```
Mounted at /content/drive
  PDFs  (x-train): 6 files in /content/drive/MyDrive/print_pdf
    Covarrubias - Tesoro lengua.pdf
    Guardiola - Tratado nobleza.pdf
    PORCONES.228.38 — 1646.pdf
    PORCONES.748.6 — 1650.pdf
    Buendia - Instruccion.pdf
    PORCONES.23.5 - 1628.pdf
  DOCXs (y-train): 7 files in /content/drive/MyDrive/Print
    PORCONES.748.6 — 1650 Transcription.docx
    PORCONES.228.38 - 1646 transcription.docx
    Covarrubias - Tesoro lengua transcription.docx
    .~lock.PORCONES.748.6 — 1650 Transcription.docx#
    Buendia - Instruccion transcription.docx
    Guardiola - Tratado nobleza transcription.docx
    PORCONES.23.5 - 1628 transcription.docx

✅ Config ready
```

```
# ── Strip common transcription suffixes for fuzzy matching ────
_SUFFIX_RE = re.compile(
    r"[-\s]*(transcription|transcript|trans|ocr|gt)[-\s]*",
    re.IGNORECASE)

def clean_stem(name: str) -> str:
    return _SUFFIX_RE.sub("", name.lower().replace(".docx","").strip())

def year_from_stem(stem: str) -> int:
    m = re.search(r"(15\d{2}|16\d{2}|17\d{2})", stem)
    return int(m.group(1)) if m else 1650

def lang_from_stem(stem: str) -> str:
    return "pt" if re.search(r"\bPT\b", stem, re.IGNORECASE) else "es"

# ── Build DOCUMENT_MAP automatically ──────────────────────────
pdf_files  = sorted(PDF_DIR.glob("*.pdf"))
docx_files = sorted(DOCX_DIR.glob("*.docx"))

if not pdf_files:
    raise FileNotFoundError(f"No PDFs found in {PDF_DIR}")
if not docx_files:
    raise FileNotFoundError(f"No DOCX files found in {DOCX_DIR}")

print(f"PDFs  found: {len(pdf_files)}")
print(f"DOCXs found: {len(docx_files)}\n")

# lookup: clean_stem → original_stem
docx_lookup = {clean_stem(d.stem): d.stem for d in docx_files}
docx_clean_keys = list(docx_lookup.keys())

DOCUMENT_MAP = {}
for pdf in pdf_files:
    pdf_key = clean_stem(pdf.stem)

    # 1 — exact clean-stem match
    if pdf_key in docx_lookup:
        matched = docx_lookup[pdf_key]
        how = "EXACT"
    else:
        # 2 — fuzzy match (cutoff 0.55)
        matches = difflib.get_close_matches(
            pdf_key, docx_clean_keys, n=1, cutoff=0.55)
        if matches:
            matched = docx_lookup[matches[0]]
            ratio   = difflib.SequenceMatcher(
                None, pdf_key, matches[0]).ratio()
            how = f"FUZZY {ratio:.2f}"
        else:
            matched = None
            how = "NO MATCH"

    DOCUMENT_MAP[pdf.stem] = {
        "pdf" : str(pdf),
        "docx": matched,
        "year": year_from_stem(pdf.stem),
        "lang": lang_from_stem(pdf.stem),
    }
    status = "✓" if matched else "✗"
    print(f"  {status} [{how}] {pdf.name}  →  {matched}.docx")

print(f"\n✅ DOCUMENT_MAP: {len(DOCUMENT_MAP)} PDF entries")
```

```
PDFs  found: 6
DOCXs found: 6

  ✓ [EXACT] Buendia - Instruccion.pdf  →  Buendia - Instruccion transcription.docx
  ✓ [EXACT] Covarrubias - Tesoro lengua.pdf  →  Covarrubias - Tesoro lengua transcription.docx
  ✓ [EXACT] Guardiola - Tratado nobleza.pdf  →  Guardiola - Tratado nobleza transcription.docx
  ✓ [FUZZY 0.95] PORCONES.228.38 — 1646.pdf  →  PORCONES.228.38 - 1646 transcription.docx
  ✓ [EXACT] PORCONES.23.5 - 1628.pdf  →  PORCONES.23.5 - 1628 transcription.docx
  ✓ [EXACT] PORCONES.748.6 — 1650.pdf  →  PORCONES.748.6 — 1650 Transcription.docx

✅ DOCUMENT_MAP: 6 PDF entries
```

```
import fitz   # PyMuPDF
from PIL import Image, ImageEnhance, ImageFilter
from pathlib import Path
import json

# ==========================================
# 1. GOOGLE DRIVE SETUP (Uncomment if in Colab)
```

```
# ==========================================
# from google.colab import drive
# drive.mount('/content/drive')
# BASE_DIR = Path('/content/drive/MyDrive/PDF_Processing')
# ==========================================

# If not using Colab, set a permanent local folder here
BASE_DIR = Path("./Saved_PDF_Data")
IMAGES_DIR = BASE_DIR / "Extracted_Images"
INDEX_FILE = BASE_DIR / "image_index.json"

MAX_SIZE  = (768, 1024)
MAX_PAGES = 10

def pdf_to_images(pdf_path: Path, out_dir: Path,
                  dpi: int = 150, max_size=MAX_SIZE,
                  max_pages: int = MAX_PAGES) -> dict:
    """Render first `max_pages` pages of a PDF to preprocessed JPEGs."""
    out_dir.mkdir(parents=True, exist_ok=True)
    doc    = fitz.open(str(pdf_path))
    zoom   = dpi / 72
    matrix = fitz.Matrix(zoom, zoom)
    result = {}

    pages_to_extract = min(len(doc), max_pages)

    for idx in range(pages_to_extract):
        pixmap = doc[idx].get_pixmap(matrix=matrix)
        img    = Image.frombytes("RGB",
                    (pixmap.width, pixmap.height), pixmap.samples)
        img.thumbnail(max_size, Image.LANCZOS)
        img    = ImageEnhance.Contrast(img).enhance(1.4)
        img    = img.filter(ImageFilter.SHARPEN)
        out    = out_dir / f"page{idx+1:04d}.jpg"
        img.save(str(out), "JPEG", quality=85)
        result[idx + 1] = out

    doc.close()
    return result


# ==========================================
# 2. EXTRACTION & CACHING LOGIC
# ==========================================
print(f"📄 Saving output to: {IMAGES_DIR}")
ALL_PAGE_IMAGES = {}

# Assuming DOCUMENT_MAP is defined elsewhere in your code
for pdf_stem, meta in DOCUMENT_MAP.items():
    pdf_path = Path(meta["pdf"])
    out_dir  = IMAGES_DIR / pdf_stem

    # Check if images already exist on Drive
    if out_dir.exists() and any(out_dir.iterdir()):
        cached = {i+1: p for i, p in enumerate(sorted(out_dir.glob("*.jpg")))}
        ALL_PAGE_IMAGES[pdf_stem] = cached
        print(f"  [skip] {pdf_stem}  ({len(cached)} pages cached on Drive)")
        continue

    pages = pdf_to_images(pdf_path, out_dir)
    ALL_PAGE_IMAGES[pdf_stem] = pages
    print(f"  ✓ {pdf_stem}  → {len(pages)} pages extracted & saved to Drive")

total = sum(len(v) for v in ALL_PAGE_IMAGES.values())
print(f"\n✅ Total pages available: {total}")

# ==========================================
# 3. SAVE THE DICTIONARY FOR LATER USE
# ==========================================
# Path objects aren't JSON serializable, so we convert them to strings
json_safe_dict = {
    stem: {str(page_num): str(path) for page_num, path in pages.items()}
    for stem, pages in ALL_PAGE_IMAGES.items()
}

with open(INDEX_FILE, "w") as f:
    json.dump(json_safe_dict, f, indent=4)

print(f"💾 Saved image index to: {INDEX_FILE}")
```

```
📄 Saving output to: Saved_PDF_Data/Extracted_Images
  ✓ Buendia - Instruccion  → 10 pages extracted & saved to Drive
```

```
  ✓ Covarrubias - Tesoro lengua  → 10 pages extracted & saved to Drive
  ✓ Guardiola - Tratado nobleza  → 10 pages extracted & saved to Drive
  ✓ PORCONES.228.38 – 1646  → 10 pages extracted & saved to Drive
  ✓ PORCONES.23.5 - 1628  → 10 pages extracted & saved to Drive
  ✓ PORCONES.748.6 – 1650  → 10 pages extracted & saved to Drive

✅ Total pages available: 60
💾 Saved image index to: Saved_PDF_Data/image_index.json
```

```
import re
import difflib
import json
from pathlib import Path
from docx import Document as DocxDoc

# ==========================================
# 1. SETUP SAVE DIRECTORY
# ==========================================
# If using Colab, ensure you have mounted Drive:
# from google.colab import drive
# drive.mount('/content/drive')
# BASE_DIR = Path('/content/drive/MyDrive/PDF_Processing')

BASE_DIR = Path("./Saved_PDF_Data") # Change this if not using Colab
BASE_DIR.mkdir(parents=True, exist_ok=True)
TRANSCRIPTIONS_FILE = BASE_DIR / "all_transcriptions.json"

# ==========================================
# 2. HELPER FUNCTIONS
# ==========================================
# Page marker pattern: "PDFp1", "PDF p2", "PDFp 3" etc.
PAGE_NUM_RE = re.compile(r"PDF\s*p\s*(\d+)", re.IGNORECASE)

def is_page_marker(text: str):
    """Return int page number if text is a standalone PDFp{N} marker."""
    m = PAGE_NUM_RE.match(text.strip())
    return int(m.group(1)) if m else None

def get_docx_path(docx_stem: str) -> Path:
    """Find the DOCX file by stem (exact first, then fuzzy fallback)."""
    exact = DOCX_DIR / f"{docx_stem}.docx"
    if exact.exists():
        return exact
    # fuzzy fallback for Drive HTML-encoding quirks
    for f in DOCX_DIR.glob("*.docx"):
        if difflib.SequenceMatcher(
                None,
                re.sub(r"[-.]", "", f.stem.lower()),
                re.sub(r"[-.]", "", docx_stem.lower())).ratio() > 0.75:
            print(f"  [fuzzy match] '{docx_stem}' → '{f.name}'")
            return f
    return exact   # will fail gracefully in parse_docx_pages

def parse_docx_pages(docx_path: Path) -> dict:
    """Parse DOCX → {page_num_int: text}."""
    doc          = DocxDoc(str(docx_path))
    pages        = {}
    current_page = None
    current_buf  = []

    for para in doc.paragraphs:
        txt      = para.text.strip()
        page_num = is_page_marker(txt)
        if page_num is not None:
            if current_page is not None and current_buf:
                pages[current_page] = "\n".join(current_buf).strip()
            current_page = page_num
            current_buf  = []
        elif txt and current_page is not None:
            current_buf.append(txt)

    if current_page is not None and current_buf:
        pages[current_page] = "\n".join(current_buf).strip()
    return pages

# ==========================================
# 3. EXTRACTION & CACHING LOGIC
# ==========================================
print("📝 Checking for cached transcriptions...")
all_transcriptions = {}

# Check if we already processed and saved this data
```

```
if TRANSCRIPTIONS_FILE.exists():
    print(f"  [CACHE FOUND] Loading from Drive: {TRANSCRIPTIONS_FILE.name}")
    with open(TRANSCRIPTIONS_FILE, "r", encoding="utf-8") as f:
        saved_data = json.load(f)

    # JSON converts integer dictionary keys to strings during save.
    # We must convert them back to integers when loading.
    for pdf_stem, pages in saved_data.items():
        all_transcriptions[pdf_stem] = {int(page_num): text for page_num, text in pages.items()}

else:
    print("  [NO CACHE] Extracting DOCX transcriptions from scratch…\n")
    for pdf_stem, meta in DOCUMENT_MAP.items():
        docx_stem = meta.get("docx")
        if not docx_stem:
            print(f"  [SKIP] {pdf_stem}  (no DOCX match)")
            continue

        docx_path = get_docx_path(docx_stem)
        if not docx_path.exists():
            print(f"  [MISSING] {docx_path.name}")
            continue

        pages = parse_docx_pages(docx_path)
        if pages:
            all_transcriptions[pdf_stem] = pages
            first_pg = next(iter(pages))
            preview  = pages[first_pg][:80].replace("\n", " ")
            print(f"  ✓ {pdf_stem}  → {len(pages)} pages")
            print(f"    Preview p{first_pg}: {preview}…\n")

    # Save the newly extracted data to Drive
    print(f"💾 Saving extracted text to: {TRANSCRIPTIONS_FILE}")
    with open(TRANSCRIPTIONS_FILE, "w", encoding="utf-8") as f:
        # ensure_ascii=False keeps special characters (like quotes/dashes) readable in the JSON file
        json.dump(all_transcriptions, f, indent=4, ensure_ascii=False)

print(f"✅ Total documents with text ready for use: {len(all_transcriptions)}")
```

```
📝 Checking for cached transcriptions...
  [NO CACHE] Extracting DOCX transcriptions from scratch…

  ✓ Buendia - Instruccion  → 3 pages
    Preview p2: Al INFINITAMENTE AMABLE NIÑO JESUS. A Vos, Dulcissimo Niño JESUS, que no solo os…

  ✓ Covarrubias - Tesoro lengua  → 3 pages
    Preview p7: CENSURA DE PEDRO DE VALENCIA CORONISTA General de el Rey nuestro Señor. POr mand…

  ✓ Guardiola - Tratado nobleza  → 3 pages
    Preview p12: POR quanto por parte de vos fray Ivan Benito Guar diola monje professo de la ord…

  ✓ PORCONES.228.38 – 1646  → 5 pages
    Preview p1: POR ANTONIO GOMEZ Oimentel, mercader, padre de Iacinto Gomez Pi- mentel. CON Doñ…

  ✓ PORCONES.23.5 - 1628  → 4 pages
    Preview p1: POR DOÑA CATALINA DE Laçarraga y Zarate, viuda de Marco Antonio de Caycedo, y el…

  ✓ PORCONES.748.6 – 1650  → 4 pages
    Preview p1: DEUS IN ADIUTORIUM MEUM INTENDE POR Don Felipe Leizado Caballero del Orden Santi…

💾 Saving extracted text to: Saved_PDF_Data/all_transcriptions.json
✅ Total documents with text ready for use: 6
```

```
import cv2
import numpy as np
import random
import json
from pathlib import Path
from PIL import Image, ImageEnhance, ImageFilter

# ==========================================
# 1. GOOGLE DRIVE SETUP & DIRECTORY CREATION
# ==========================================
# If using Colab, ensure you have mounted Drive:
# from google.colab import drive
# drive.mount('/content/drive')
# BASE_DIR = Path('/content/drive/MyDrive/PDF_Processing')

BASE_DIR = Path("./Saved_PDF_Data") # Change this to your Drive path if in Colab
DATASET_DIR = BASE_DIR / "Training_Dataset"

ORIG_IMG_DIR = DATASET_DIR / "x_train_images"
AUG_IMG_DIR  = DATASET_DIR / "x_train_images_aug"
LABELS_BASE  = DATASET_DIR / "y_train_labels.json"
```

```
LABELS_AUG   = DATASET_DIR / "y_train_labels_aug.json"

# Create directories on Drive if they don't exist
ORIG_IMG_DIR.mkdir(parents=True, exist_ok=True)
AUG_IMG_DIR.mkdir(parents=True, exist_ok=True)

# ==========================================
# 2. AUGMENTATION HELPERS
# ==========================================
def _rotate(img: np.ndarray, lo=-10.0, hi=10.0) -> np.ndarray:
    angle = random.uniform(lo, hi)
    h, w  = img.shape[:2]
    M     = cv2.getRotationMatrix2D((w/2, h/2), angle, 1.0)
    return cv2.warpAffine(img, M, (w, h),
                          flags=cv2.INTER_LANCZOS4,
                          borderMode=cv2.BORDER_REPLICATE)

def _crop(img: np.ndarray, lo=0.85, hi=0.97) -> np.ndarray:
    h, w  = img.shape[:2]
    fw, fh = random.uniform(lo, hi), random.uniform(lo, hi)
    cw, ch = max(1, int(w*fw)), max(1, int(h*fh))
    x0 = random.randint(0, max(0, w-cw))
    y0 = random.randint(0, max(0, h-ch))
    return cv2.resize(img[y0:y0+ch, x0:x0+cw], (w, h),
                      interpolation=cv2.INTER_LANCZOS4)

def _contrast(img: np.ndarray, factor=None) -> np.ndarray:
    f   = factor or random.uniform(1.2, 1.6)
    pil = Image.fromarray(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    pil = ImageEnhance.Contrast(pil).enhance(f)
    pil = pil.filter(ImageFilter.SHARPEN)
    return cv2.cvtColor(np.array(pil), cv2.COLOR_RGB2BGR)

def augment_page(src: Path, dst_dir: Path, n=3, crop_p=0.70) -> list:
    # Read image from Drive/Local
    img = cv2.imread(str(src))
    if img is None: return []
    saved = []
    for v in range(n):
        aug = _rotate(img.copy()); ops = ["rot"]
        if random.random() < crop_p: aug = _crop(aug);  ops.append("crop")
        aug = _contrast(aug);  ops.append("contrast")
        fname = f"{src.stem}_aug{v}_{'_'.join(ops)}.jpg"
        out   = dst_dir / fname
        # Write augmented image back to Drive
        cv2.imwrite(str(out), aug, [cv2.IMWRITE_JPEG_QUALITY, 85])
        saved.append(str(out))
    return saved

# ==========================================
# 3. MAIN PAIRING LOOP & CACHE CHECK
# ==========================================
base_pairs, aug_pairs = [], []

# Check if we already built this dataset on a previous run
if LABELS_AUG.exists() and LABELS_BASE.exists():
    print(f"📦  [CACHE FOUND] Loading existing paired dataset from Drive...")
    with open(LABELS_BASE, "r", encoding="utf-8") as f:
        base_pairs = json.load(f)
    with open(LABELS_AUG, "r", encoding="utf-8") as f:
        aug_pairs = json.load(f)

else:
    print("📦  Building paired dataset from scratch and saving to Drive…\n")
    # Note: Requires 'all_transcriptions' and 'ALL_PAGE_IMAGES' from previous steps
    for pdf_stem, pages in all_transcriptions.items():
        page_images = ALL_PAGE_IMAGES.get(pdf_stem, {})
        if not page_images:
            print(f"  [SKIP] {pdf_stem} — no extracted images"); continue
        print(f"  {pdf_stem}")
        matched = 0

        for pg_num, text in pages.items():
            text = text.strip()
            if not text: continue
            src_img = page_images.get(pg_num)
            if src_img is None or not src_img.exists():
                print(f"    ✗ missing page{pg_num:04d}.jpg"); continue

            # copy original to dataset dir
            safe   = pdf_stem.replace(" ", "_")
            o_path = ORIG_IMG_DIR / f"{safe}_page{pg_num:04d}.jpg"
```

```
            if not o_path.exists():
                cv = cv2.imread(str(src_img))
                if cv is not None:
                    cv2.imwrite(str(o_path), cv, [cv2.IMWRITE_JPEG_QUALITY, 88])

            entry = {"xtrain_image": str(o_path), "ytrain_text": text,
                     "metadata": {"doc": pdf_stem, "page": pg_num,
                                  "augmented": False}}
            base_pairs.append(entry)
            aug_pairs.append(entry)

            for ap in augment_page(src_img, AUG_IMG_DIR):
                aug_pairs.append({
                    "xtrain_image": ap, "ytrain_text": text,
                    "metadata": {"doc": pdf_stem, "page": pg_num,
                                 "augmented": True}})
            matched += 1
        print(f"    ✓ {matched} pages paired\n")

    # Filter out any broken paths
    base_pairs = [s for s in base_pairs if Path(s["xtrain_image"]).exists()]
    aug_pairs  = [s for s in aug_pairs  if Path(s["xtrain_image"]).exists()]

    # Save the JSON label mappings to Drive
    with open(LABELS_BASE,"w",encoding="utf-8") as f:
        json.dump(base_pairs, f, indent=2, ensure_ascii=False)
    with open(LABELS_AUG,"w",encoding="utf-8") as f:
        json.dump(aug_pairs, f, indent=2, ensure_ascii=False)

print("="*55)
print(f"  Original pairs     : {len(base_pairs)}")
print(f"  After augmentation : {len(aug_pairs)}")
print(f"  Orig  labels saved → {LABELS_BASE}")
print(f"  Aug   labels saved → {LABELS_AUG}")
print("="*55)
```

```
🗂  Building paired dataset from scratch and saving to Drive…

  Buendia - Instruccion
    ✓ 3 pages paired

  Covarrubias - Tesoro lengua
    ✓ 3 pages paired

  Guardiola - Tratado nobleza
    ✗ missing page0012.jpg
    ✗ missing page0013.jpg
    ✗ missing page0014.jpg
    ✓ 0 pages paired

  PORCONES.228.38 – 1646
    ✓ 5 pages paired

  PORCONES.23.5 - 1628
    ✓ 4 pages paired

  PORCONES.748.6 – 1650
    ✓ 4 pages paired

=======================================================
  Original pairs     : 19
  After augmentation : 76
  Orig  labels saved → Saved_PDF_Data/Training_Dataset/y_train_labels.json
  Aug   labels saved → Saved_PDF_Data/Training_Dataset/y_train_labels_aug.json
=======================================================
```

```
from unsloth import FastVisionModel

MODEL_ID = "unsloth/Qwen2.5-VL-7B-Instruct-bnb-4bit"
print(f"🔄 Loading {MODEL_ID} …")

model, tokenizer = FastVisionModel.from_pretrained(
    model_name                 = MODEL_ID,
    load_in_4bit               = True,
    use_gradient_checkpointing = "unsloth",
)

model = FastVisionModel.get_peft_model(
    model,
    r                          = 16,
    lora_alpha                 = 32,
    lora_dropout               = 0.05,
    bias                       = "none",
    random_state               = SEED,
```

```
    use_rslora                 = False,
    finetune_vision_layers     = True,
    finetune_language_layers   = True,
    finetune_attention_modules = True,
    finetune_mlp_modules       = True,
    target_modules             = ["q_proj","k_proj","v_proj","o_proj",
                                  "gate_proj","up_proj","down_proj"],
)
model.print_trainable_parameters()
print("✅ Model + LoRA ready")
```

```
import json
import random
from pathlib import Path
from PIL import Image
from tqdm import tqdm
from datasets import Dataset, load_from_disk

# ==========================================
# 1. SETUP DRIVE PATHS
# ==========================================
BASE_DIR = Path("./Saved_PDF_Data") # Change to your Drive path if in Colab
DATASET_DIR = BASE_DIR / "Training_Dataset"

LABELS_AUG   = DATASET_DIR / "y_train_labels_aug.json"
LABELS_BASE  = DATASET_DIR / "y_train_labels.json"

# Paths where we will save the finalized Hugging Face datasets
TRAIN_DS_PATH = DATASET_DIR / "hf_train_dataset"
VAL_DS_PATH   = DATASET_DIR / "hf_val_dataset"

# ==========================================
# 2. PROMPT CONSTANTS
# ==========================================
SYSTEM_MSG = (
    "You are an expert OCR system specialising in 17th-century Spanish "
    "printed sources (siglos XVI–XVII). Transcribe ALL visible printed text "
    "exactly as it appears, preserving original Early Modern Spanish spelling, "
    "punctuation, abbreviations, and line structure. Disregard decorative "
    "borders, stamps, and marginalia."
)
OCR_INSTRUCTION = (
    "Carefully transcribe every word of printed text visible in this "
    "17th-century Spanish document page. Preserve the original orthography "
    "faithfully — do NOT modernise spelling. Output only the transcription."
)

MAX_IMG_SIZE = (768, 768)

def convert_to_conversation(sample: dict) -> dict:
    img = Image.open(sample["xtrain_image"]).convert("RGB")
    img.thumbnail(MAX_IMG_SIZE, Image.Resampling.LANCZOS)
    return {"messages": [
        {"role": "system",
         "content": [{"type": "text", "text": SYSTEM_MSG}]},
        {"role": "user",
         "content": [{"type": "image", "image": img},
                     {"type": "text",  "text": OCR_INSTRUCTION}]},
        {"role": "assistant",
         "content": [{"type": "text", "text": sample["ytrain_text"]}]},
    ]}

# ==========================================
# 3. DATASET GENERATION & CACHING
# ==========================================
# Check if the finalized dataset already exists on Drive
if TRAIN_DS_PATH.exists() and VAL_DS_PATH.exists():
    print("📂  [CACHE FOUND] Loading finalized datasets directly from Drive...")
```

```
    train_ds = load_from_disk(str(TRAIN_DS_PATH))
    val_ds   = load_from_disk(str(VAL_DS_PATH))
    print(f"  ✅ Loaded {len(train_ds)} train | {len(val_ds)} val conversations")

else:
    print("🗃  Building conversation dataset from scratch...")
    LABELS_PATH = LABELS_AUG if LABELS_AUG.exists() else LABELS_BASE
    print(f"📂 Loading: {LABELS_PATH.name}")

    with open(LABELS_PATH,"r",encoding="utf-8") as f:
        all_pairs = json.load(f)

    all_pairs = [s for s in all_pairs
                 if Path(s["xtrain_image"]).exists() and s["ytrain_text"].strip()]
    print(f"  ✅ Valid samples: {len(all_pairs)}")

    random.seed(42) # Set seed for reproducible shuffling
    random.shuffle(all_pairs)
    val_n     = max(2, int(0.15 * len(all_pairs)))
    val_raw   = all_pairs[:val_n]
    train_raw = all_pairs[val_n:]
    print(f"  Train: {len(train_raw)}  |  Val: {len(val_raw)}")

    # Convert to conversation format
    print("\n🔄 Converting to conversation format (resizing images) …")
    train_conv = [convert_to_conversation(s) for s in tqdm(train_raw, desc="Train")]
    val_conv   = [convert_to_conversation(s) for s in tqdm(val_raw,   desc="Val")]

    # Wrap in Hugging Face Dataset objects (this automatically handles the PIL images)
    print("\n📦 Packing into Hugging Face Datasets...")
    train_ds = Dataset.from_list(train_conv)
    val_ds   = Dataset.from_list(val_conv)

    # Save to Drive
    print(f"💾 Saving to Drive at: {DATASET_DIR.name} ...")
    train_ds.save_to_disk(str(TRAIN_DS_PATH))
    val_ds.save_to_disk(str(VAL_DS_PATH))

    print(f"✅ {len(train_ds)} train | {len(val_ds)} val conversations saved & ready!")

# Sanity Check (works whether loaded from cache or built fresh)
print("\n🔍 Sanity Check on Sample 0:")
sample_msg = train_ds[0]["messages"]
print(f"  System : {sample_msg[0]['content'][0]['text'][:60]}...")
print(f"  Image  : {type(sample_msg[1]['content'][0]['image'])} (Loaded successfully)")
print(f"  Target : {sample_msg[2]['content'][0]['text'][:60]}...")
```

```
import time
import torch
from pathlib import Path
from trl import SFTTrainer, SFTConfig
from unsloth.trainer import UnslothVisionDataCollator
from unsloth import FastVisionModel

# ==========================================
# 0. DYNAMIC HARDWARE DETECTION
# ==========================================
HAS_GPU = torch.cuda.is_available()
IS_BF16 = False
USE_CPU = not HAS_GPU
```

```
if HAS_GPU:
    # Check compute capability (Ampere GPUs are 8.x and higher)
    major, _ = torch.cuda.get_device_capability()
    if major >= 8:
        IS_BF16 = True
        print("🟢 High-end GPU Detected (Ampere+). Using fast bf16 precision.")
    else:
        print("🟡 Standard GPU Detected (e.g., T4). Using fp16 precision.")
else:
    print("🔴 No GPU Detected! Falling back to CPU.")
    print("⚠️ WARNING: Training a 7B model on CPU is not recommended and will take a very long time.")

# ==========================================
# 1. SETUP DRIVE PATHS
# ==========================================
BASE_DIR = Path("./Saved_PDF_Data") # Change to your Drive path
CHECKPOINTS_DIR = BASE_DIR / "Training_Checkpoints"
FINAL_ADAPTER_DIR = BASE_DIR / "My_Best_Qwen_LoRA"

# ==========================================
# 2. CACHE CHECK & TRAINING LOGIC
# ==========================================
if FINAL_ADAPTER_DIR.exists() and (FINAL_ADAPTER_DIR / "adapter_config.json").exists():
    print(f"✅ [CACHE FOUND] Best model already trained and saved at: {FINAL_ADAPTER_DIR}")
    print("Skipping training step. You are ready for inference!")

else:
    print("⚙️ Preparing model for training...")
    FastVisionModel.for_training(model)

    sft_args = SFTConfig(
        output_dir                  = str(CHECKPOINTS_DIR),
        num_train_epochs            = 8,
        per_device_train_batch_size = 1,
        per_device_eval_batch_size  = 1,
        gradient_accumulation_steps = 4,
        warmup_steps                = 5,
        learning_rate               = 2e-4,

        # --- DYNAMIC HARDWARE CONFIG ---
        fp16                        = HAS_GPU and not IS_BF16,
        bf16                        = HAS_GPU and IS_BF16,
        use_cpu                     = USE_CPU,
        # -------------------------------

        logging_steps               = 5,
        eval_strategy               = "epoch",
        save_strategy               = "epoch",
        save_total_limit            = 2,
        load_best_model_at_end      = True,
        metric_for_best_model       = "eval_loss",
        optim                       = "adamw_bnb_8bit" if HAS_GPU else "adamw_torch", # 8bit optim requires GPU
        weight_decay                = 0.01,
        lr_scheduler_type           = "cosine",
        seed                        = SEED, # Ensure SEED is defined earlier in your script
        remove_unused_columns       = False,
        dataset_text_field          = "",
        dataset_kwargs              = {"skip_prepare_dataset": True},
        dataset_num_proc            = 1,
        max_seq_length              = 1024,
        report_to                   = "none",
    )

    trainer = SFTTrainer(
        model         = model,
        tokenizer     = tokenizer,
        data_collator = UnslothVisionDataCollator(model, tokenizer),
        train_dataset = train_ds,
        eval_dataset  = val_ds,
        args          = sft_args,
    )

    t0 = time.time()
    print("🚀 Training started …")
    trainer.train()

    elapsed    = time.time() - t0
    final_loss = trainer.state.log_history[-1].get(
                    "train_loss",
                    trainer.state.log_history[-1].get("loss", float("nan")))

    print(f"\n✅ Training Done — {elapsed:.1f}s | final loss = {final_loss:.4f}")
```

```
    # ==========================================
    # 3. SAVE THE BEST MODEL
    # ==========================================
    print(f"💾 Saving the BEST model (lowest eval loss) to Drive at: {FINAL_ADAPTER_DIR} ...")
    model.save_pretrained(str(FINAL_ADAPTER_DIR))
    tokenizer.save_pretrained(str(FINAL_ADAPTER_DIR))
    print("🎉 All done! Your custom 17th-century Spanish OCR model is permanently saved.")
```

```
import shutil
from pathlib import Path
from google.colab import drive

# 1. Mount your Google Drive (it will ask for permission if not already mounted)
drive.mount('/content/drive')

# 2. Define your source and destination paths
source_dir = Path("/content/Saved_PDF_Data")
drive_dest_dir = Path("/content/drive/MyDrive/Saved_PDF_Data")

# 3. Copy the entire folder over
print(f"🔄 Copying all data from {source_dir} to {drive_dest_dir}...")

if source_dir.exists():
    # dirs_exist_ok=True ensures it overwrites/updates existing files
    # rather than crashing if you run this script more than once.
    shutil.copytree(source_dir, drive_dest_dir, dirs_exist_ok=True)
    print("✅ Backup complete! Your data is now safely stored on Google Drive.")
else:
    print(f"🔴 Error: The folder {source_dir} does not exist. Check the path and try again.")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force
🔄 Copying all data from /content/Saved_PDF_Data to /content/drive/MyDrive/Saved_PDF_Data...
✅ Backup complete! Your data is now safely stored on Google Drive.
```

```
import gc
import torch
from pathlib import Path
from unsloth import FastVisionModel

# ── Paths ──────────────────────────────────────────────────────────
BASE_DIR          = Path("/content/drive/MyDrive/Saved_PDF_Data")  # ← Drive path
CHECKPOINTS_DIR   = BASE_DIR / "Training_Checkpoints"
FINAL_ADAPTER_DIR = BASE_DIR / "My_Best_Qwen_LoRA"

# ── Hardware detection ─────────────────────────────────────────────
```

```
HAS_GPU = torch.cuda.is_available()
IS_BF16 = False
if HAS_GPU:
    major, _ = torch.cuda.get_device_capability()
    IS_BF16   = major >= 8
    print("🟢 Ampere+ GPU — bf16" if IS_BF16 else "🟡 T4 GPU — fp16")
else:
    print("🔴 No GPU detected.")

# ── Clear VRAM before loading ──────────────────────────────────────
for var in ("model", "tokenizer"):
    try:    del globals()[var]
    except KeyError: pass
torch.cuda.empty_cache()
gc.collect()

# ═══════════════════════════════════════════════════════════════════════
#  CACHE CHECK  →  load if saved, train only if missing
# ═══════════════════════════════════════════════════════════════════════
adapter_config = FINAL_ADAPTER_DIR / "adapter_config.json"

if FINAL_ADAPTER_DIR.exists() and adapter_config.exists():
    # ── ✅ Already trained — just load for inference ──────────────────
    print(f"\n✅ Trained adapter found at: {FINAL_ADAPTER_DIR}")
    print("⏭️  Skipping training. Loading directly for inference...\n")

    model, tokenizer = FastVisionModel.from_pretrained(
        model_name   = str(FINAL_ADAPTER_DIR),
        load_in_4bit = True,
        dtype        = torch.float16,   # fp16 — safe on T4, fixes dtype crash
    )
    FastVisionModel.for_inference(model)
    print("✅ Model + tokenizer loaded and ready for inference!")

else:
    # ── ⚙️ Not yet trained — run training ────────────────────────────
    print(f"\n⚙️ No saved adapter found at: {FINAL_ADAPTER_DIR}")
    print("Starting training from scratch...\n")

    import time
    from trl import SFTTrainer, SFTConfig
    from unsloth.trainer import UnslothVisionDataCollator

    SEED = 42
    FastVisionModel.for_training(model)

    sft_args = SFTConfig(
        output_dir                  = str(CHECKPOINTS_DIR),
        num_train_epochs            = 8,
        per_device_train_batch_size = 1,
        per_device_eval_batch_size  = 1,
        gradient_accumulation_steps = 4,
        warmup_steps                = 5,
        learning_rate               = 2e-4,
        fp16                        = HAS_GPU and not IS_BF16,
        bf16                        = HAS_GPU and IS_BF16,
        use_cpu                     = not HAS_GPU,
        logging_steps               = 5,
        eval_strategy               = "epoch",
        save_strategy               = "epoch",
        save_total_limit            = 2,
        load_best_model_at_end      = True,
        metric_for_best_model       = "eval_loss",
        optim                       = "adamw_bnb_8bit" if HAS_GPU else "adamw_torch",
        weight_decay                = 0.01,
        lr_scheduler_type           = "cosine",
        seed                        = SEED,
        remove_unused_columns       = False,
        dataset_text_field          = "",
        dataset_kwargs              = {"skip_prepare_dataset": True},
        dataset_num_proc            = 1,
        max_seq_length              = 1024,
        report_to                   = "none",
    )

    trainer = SFTTrainer(
        model         = model,
        tokenizer     = tokenizer,
        data_collator = UnslothVisionDataCollator(model, tokenizer),
        train_dataset = train_ds,
        eval_dataset  = val_ds,
        args          = sft_args,
```

```
    )

    t0 = time.time()
    print("🚀 Training started...")
    trainer.train()

    elapsed    = time.time() - t0
    final_loss = trainer.state.log_history[-1].get(
        "train_loss",
        trainer.state.log_history[-1].get("loss", float("nan")))
    print(f"\n✅ Training Done — {elapsed:.1f}s | final loss = {final_loss:.4f}")

    # Save best model
    print(f"💾 Saving to: {FINAL_ADAPTER_DIR}")
    FINAL_ADAPTER_DIR.mkdir(parents=True, exist_ok=True)
    model.save_pretrained(str(FINAL_ADAPTER_DIR))
    tokenizer.save_pretrained(str(FINAL_ADAPTER_DIR))

    # Switch to inference mode after training
    FastVisionModel.for_inference(model)
    print("🎉 Training complete. Model ready for inference!")
```

```
# Optional: use more samples for a richer evaluation
# Uses ALL 19 pairs as val (since test set is tiny, full eval makes more sense)
val_raw = all_pairs   # evaluate on everything
print(f"✅ Evaluating on all {len(val_raw)} available pairs")
```

```
✅ Evaluating on all 19 available pairs
```

```
# Optional: 30% val split → ~5-6 samples
random.seed(42)
random.shuffle(all_pairs)
val_n   = max(4, int(0.30 * len(all_pairs)))  # ~5 samples
val_raw = all_pairs[:val_n]
print(f"✅ val_raw: {len(val_raw)} samples")
```

```
✅ val_raw: 5 samples
```

```
import json, random
from pathlib import Path

BASE_DIR    = Path("/content/drive/MyDrive/Saved_PDF_Data")
DATASET_DIR = BASE_DIR / "Training_Dataset"

# ── Step 1: Find correct label file (tries all naming variants) ──────
label_candidates = [
    DATASET_DIR / "y_train_labels_aug.json",   # preferred (augmented)
    DATASET_DIR / "y_train_labels.json",        # fallback (base)
    DATASET_DIR / "ytrain_labels_aug.json",     # old naming
    DATASET_DIR / "ytrain_labels.json",         # old naming
]
LABELS_PATH = next((p for p in label_candidates if p.exists()), None)

if LABELS_PATH is None:
    print("❌ No label file found. Listing Training_Dataset contents:")
    for f in sorted(DATASET_DIR.iterdir()):
        print(f"   {f.name}")
    raise FileNotFoundError("Cannot find label JSON — check output above.")

print(f"✅ Found label file: {LABELS_PATH.name}")

# ── Step 2: Load all pairs from JSON ──────────────────────────────────
with open(LABELS_PATH, "r", encoding="utf-8") as f:
    all_pairs = json.load(f)
```

```
print(f"   Total JSON entries : {len(all_pairs)}")

# ── Step 3: Index every image on Drive by filename ──────────────────
# (the stored paths point to old /content/ paths — we remap by filename)
img_lookup = {}
for folder_name in ("x_train_images", "x_train_images_aug",
                    "xtrain_images",   "xtrain_images_aug"):
    folder = DATASET_DIR / folder_name
    if folder.exists():
        found = list(folder.glob("*.jpg"))
        for f in found:
            img_lookup[f.name] = str(f)
        print(f"   Indexed {len(found):>4} images  ← {folder_name}/")

print(f"   Total images indexed: {len(img_lookup)}")

if not img_lookup:
    print("\n❌ No images found! Listing Training_Dataset:")
    for f in sorted(DATASET_DIR.iterdir()):
        print(f"   {f.name}")
    raise FileNotFoundError("Image folders are empty or missing.")

# ── Step 4: Remap stored paths → real Drive paths ────────────────────
def remap(stored_path: str) -> str:
    return img_lookup.get(Path(stored_path).name, "")

valid_pairs = []
for s in all_pairs:
    drive_path = remap(s["xtrain_image"])
    if drive_path and s["ytrain_text"].strip():
        valid_pairs.append({
            "xtrain_image": drive_path,
            "ytrain_text":  s["ytrain_text"],
        })

print(f"\n   Valid pairs after remapping: {len(valid_pairs)}")

# Diagnostics if remapping failed
if len(valid_pairs) == 0:
    print("\n⚠️  Zero valid pairs! Showing stored paths vs Drive lookup:")
    for s in all_pairs[:3]:
        fname  = Path(s["xtrain_image"]).name
        result = img_lookup.get(fname, "NOT FOUND")
        print(f"   stored  : {s['xtrain_image']}")
        print(f"   fname   : {fname}")
        print(f"   on Drive: {result}")
    raise ValueError("Path remapping failed — see diagnostics above.")

# ── Step 5: Reproduce the exact training split (seed=42, 15% val) ────
random.seed(42)
random.shuffle(valid_pairs)

val_n   = max(2, int(0.15 * len(valid_pairs)))
val_raw = valid_pairs[:val_n]

print(f"\n✅ val_raw ready — {len(val_raw)} samples")
print(f"   Example image : {Path(val_raw[0]['xtrain_image']).name}")
print(f"   Example text  : {val_raw[0]['ytrain_text'][:80]}...")
```

```
✅ Found label file: y_train_labels_aug.json
   Total JSON entries : 76
   Indexed   19 images  ← x_train_images/
   Indexed   32 images  ← x_train_images_aug/
   Total images indexed: 51

   Valid pairs after remapping: 76

✅ val_raw ready — 11 samples
   Example image : page0003_aug0_rot_crop_contrast.jpg
   Example text  : der seis mil ducados, que auian tocado a su hijo Ia-
cinto Gomez por su legitima...
```

```
# Force dynamic cache — no reload needed, model stays in VRAM
model.generation_config.cache_implementation = "dynamic"
print("✅ Cache patched to dynamic (fp16-safe)")
```

```
✅ Cache patched to dynamic (fp16-safe)
```

```
!pip install nbformat
```

```
Requirement already satisfied: nbformat in /usr/local/lib/python3.12/dist-packages (5.10.4)
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.12/dist-packages (from nbformat) (2
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.12/dist-packages (from nbformat) (4.26.0
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in /usr/local/lib/python3.12/dist-packages (from nbform
Requirement already satisfied: traitlets>=5.1 in /usr/local/lib/python3.12/dist-packages (from nbformat) (5.7.1)
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nb
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.12/dist-packages (from jupyter-core!=5
Requirement already satisfied: typing-extensions>=4.4.0 in /usr/local/lib/python3.12/dist-packages (from referenc
```

```
import nbformat

# load notebook
nb = nbformat.read("step_1_model_training.ipynb", as_version=4)

# remove broken widget metadata
if "widgets" in nb.metadata:
    del nb.metadata["widgets"]

# save fixed notebook
nbformat.write(nb, "step_1_model_training.ipynb")

print("Fixed notebook saved!")
```